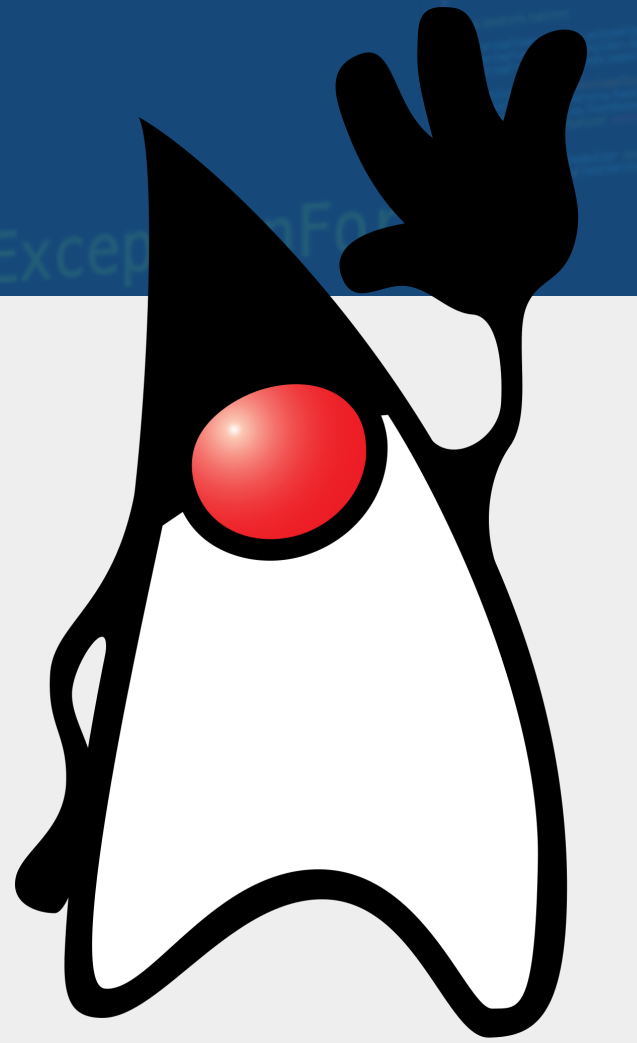


Java Programlama Dili

9. Bölüm İşlemciler

Akın Kaldıroğlu



Amaç



- Bu bölümde işlemciler (operators) ele alınacaktır

Konular



- İşlemci Nedir?
- Atama
- Aritmetik İşlemciler
 - İkili Aritmetik İşlemciler
 - Tekli Aritmetik İşlemciler
- İlişkisel (Kıyas) İşlemcileri
- Mantıksal İşlemcileri
- Şartlı İşlemciler
- Bit İşlemcileri
 - Mantıksal Bit İşlemcileri
 - Kaydırma İşlemcileri
- Diğer İşlemciler
 - Not
- Öncelik ve Değerlendirme Sırası
- Sayısal Yükseltmeler
- İşlemci Çoklu Kullanımı
- Ödevler
- Bundan Sonra?

İşlemci Nedir?



İşlemci (Operator)



- **İşlemci** ya da **operatör (operator)**, bir, iki ya da üç tane **işlenen/girdi (input/operand)** üzerinde işlem yapıp bir sonuç üreten bir semboldür
 - İşlenen bir ifade (expression) de olabilir
- İşlemciler, muhtemelen programlamadaki en sıkıcı ve en çok hata yapılan konudur

Tüm İşlemciler



- Java'da 38 tane işlemci sembolü (token) vardır:

= > < ! ~ ? : ->

== <= >= != && || ++ --

+ - * / & | ^ % << >> >>>

+= -= *= /= &= |= ^= %= <<= >>= >>>=

İşlemci Türleri



- İşlemciler, işlediği girdi sayısı ya da işlevi açısından sınıflandırılabilir
- İşlenen girdi sayısı açısından: Bir, iki ya da üç işlenen alan işlemciler
- İşlevi açısından: Aritmetik (arithmetic), ilişkisel/kıyaslama (relational/comparison), mantıksal (logical), bit (bitwise) işlemciler

Tekli, İkili ve Üçlü İşlemciler



- Tek işlenenli işlemcilere, **tekli işlemci (unary operator)**

`operator operand` ya da `operand operator`

`++i` ya da `i++`

- İki işlenenli işlemcilere, **ikili işlemci (binary operator)**,

`operand1 operator operand2`

`2 + 5` ya da `true & false`

- Üç işlenenli işlemcilere, **üçlü işlemci (ternary operator)** denir

`operand1 ? operand2 : operand3`

`int i = j > k ? j : k`

İşlevlerine Göre İşlemciler - I



- **Aritmetik (arithmetic) işlemciler** 9 tanedir.
- 5 tanesi matematiksel işlemlerde kullanılır: **toplama +**, **çıkarma -**, **çarpma ***, **bölme /** ve **mod (kalan) %**
- Bu aritmetik işlemciler iki işlenen alır ve bir sayı sonuç üretirler
- Ayrıca tek işlenen alan 4 aritmetik işlemci daha vardır: **arttırma ++**, **eksiltme --**, **artı +** ve **eksi -**
- Bu işlemciler de bir sayı sonuç üretirler

İşlevlerine Göre İşlemciler - II



- **İlişkisel (relational) veya kıyaslama (comparison)** işlemcileri 6 tanedir ve daima `boolean` bir değer üretir: **Eşit mi** `==`, **eşit değil mi** `!=`, **büyük mü** `>`, **küçük mü** `<`, **büyük ya da eşit mi** `>=` ve **küçük ya da eşit mi** `<=`
- **Mantıksal (logical)** işlemciler 6 tanedir ve daima `boolean` işlenenler alır ve `boolean` bir değer üretir: **VE (AND)** `&`, **VEYA (OR)** `|`, **TÜMELEYEN/DEĞİL (NOT)** `!` ve **DIŞLAYICI (XOR)** `^`
- Ayrıca `&` ve `|`'nin kısa devre (short circuit) halleri de vardır: `&&` ve `||`

İşlevlerine Göre İşlemciler - II



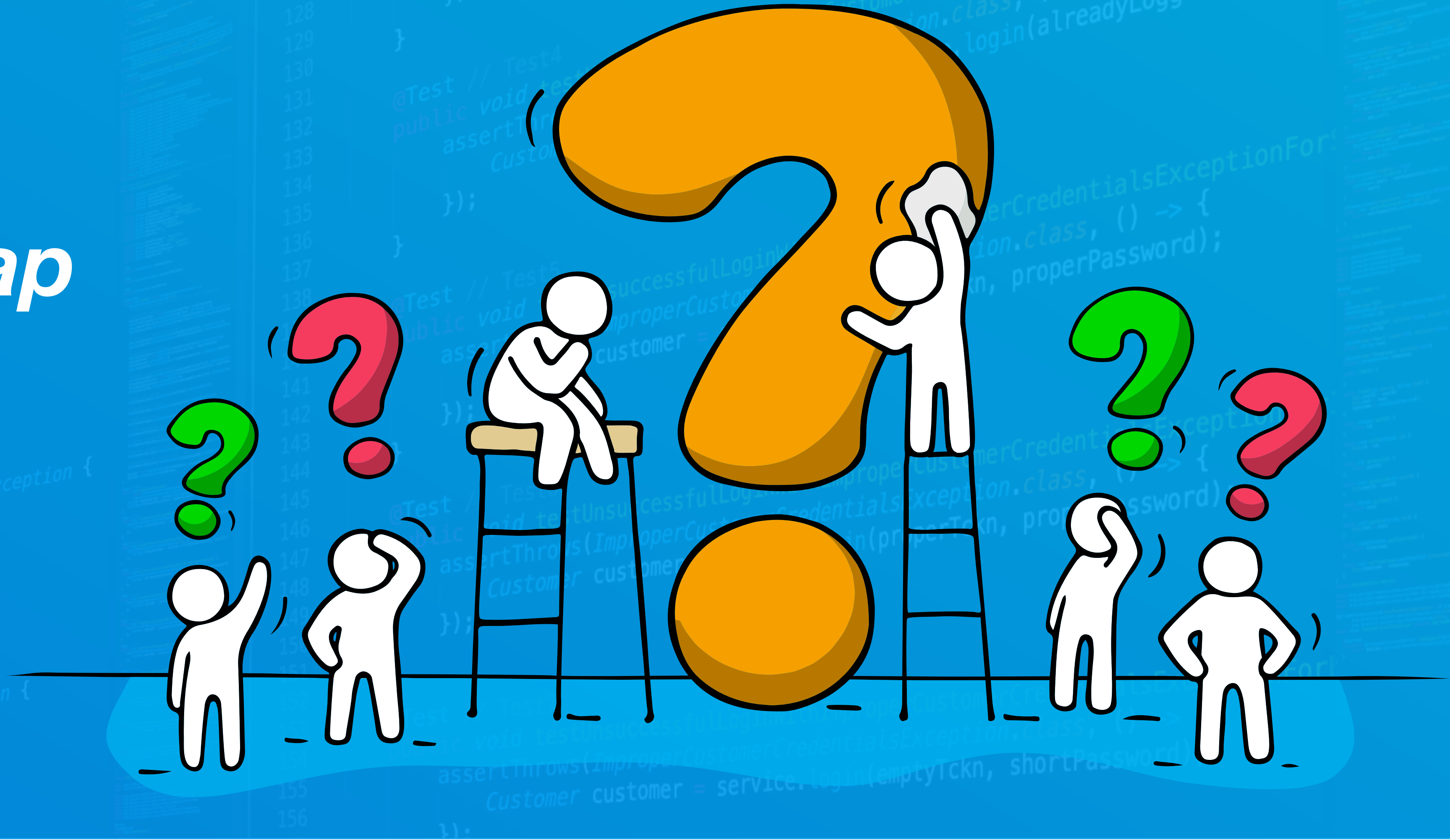
- Şartlı (conditional) işlemciler 3 tanedir: Şartlı VE (Conditional AND) &&, Şartlı VEYA (Conditional OR) ||, Üçlü Şart (Ternary Conditional) ? :
- Bunlardan && ve ||'yi & ve |'un kısa devre hali olarak daha önce görmüştük, sağ tarafını, soldakinin değerine bağlı olarak işlediği için şartlı (conditional) olarak adlandırılır
- Diğer şartlı işlemci, üçlü işlemcidir (ternary operator): ? :

İşlevlerine Göre İşlemciler - IV



- **Bit (bitwise)** işlemciler 7 tanedir ve sayısal işlenenlerin bitleri üzerine işlem yapıp yine sayı üretirler: **VE (AND) &**, **VEYA (OR) |**, **TÜMELEYEN (COMPLEMENT) ~**, **Dışlayıcı (XOR) ^** ve 3 tane **kaydırma işlemcisi (shift operator)**: **<<**, **>>**, ve **>>>**
- **Atama (assignment)** işlemcisi aslen bir tanedir: **=**
 - Fakat, toplama **+**, çıkarma **-** gibi aritmetik işlemcilerle **+=**, **-=** ya da **<<=** gibi kaydırma işlemcileriyle birlikte kullanılan 11 tane bileşik atama (compound assignment) işlemcisi daha vardır

Soru ve Cevap Zamanı!



Atama



Atama (Assignment) - I



- **Atama (assignment) işlemcisi, =**, en basit işlemcidir
- İki işlenen alır ve sağdaki işleneni (tek bir değer üreten daha karmaşık bir ifade de olabilir) soldaki işlenene atar
- Atama bazen değer bazen de nesne atamasıdır

```
//Değer ataması
int i = 5;
boolean b = false;

int k = 30, t = 5;
//İfadenin ürettiği değerın atanması
int j = 2 * i + k / t;

//Nesne ataması
Car myCar = new Car();
```

Atama (Assignment) - II



- Atama bir ifadedir (expression) ve sonucu kendisine atama yapılan sol taraftaki (lvalue) değişkenin atamadan sonraki değeridir

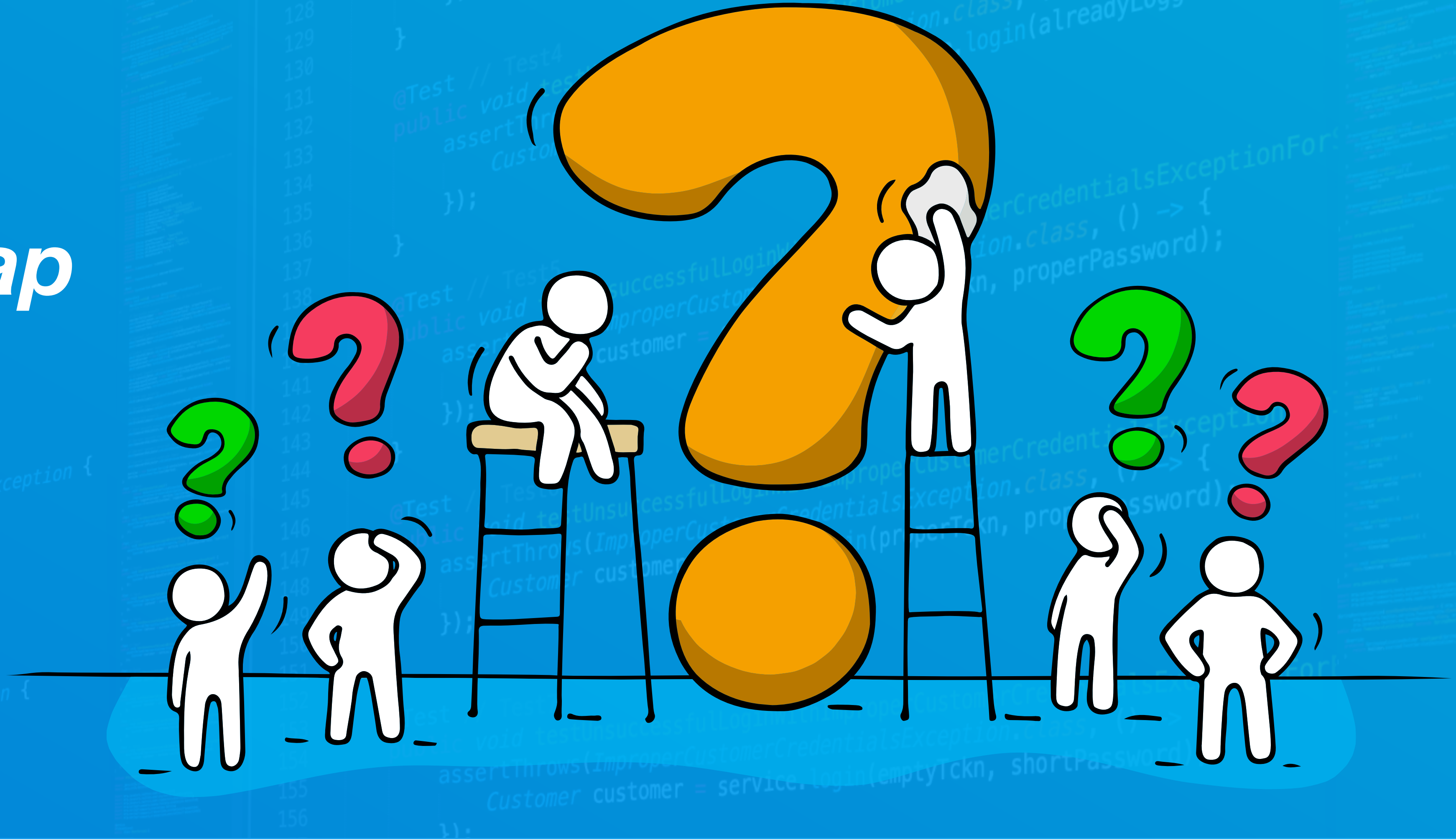
```
boolean b1;  
b1 = true; // İfadenin değeri true
```

- Dolayısıyla aşağıdaki gibi ifadeler yazılabilir:

```
boolean b1, b2;  
b1 = (b2 = true);  
b1 = b2 = true;
```

- Atamanın, += gibi bileşik (compound assignment) hallerini birazdan göreceğiz

Soru ve Cevap Zamanı!



Aritmetik İşlemler



Aritmetik İşlemciler



- Aritmetik işlemciler tam veya virgüllü sayı tipinden değerleri işleyen ve yine tam veya virgüllü sayı değer üreten işlemcilerdir
- Aritmetik işlemciler ikili veya tekli olabilir
- Önce ikilileri görelim sonra teklileri ele alalım



İkili Aritmetik İşlemciler

İkili Aritmetik İşlemciler - I



- İkili aritmetik işlemciler (binary arithmetic operators) 5 tanedir:
 - **+**: Toplama (addition)
 - **-**: Çıkarma (subtraction)
 - *****: Çarpma (multiplication)
 - **/**: Bölme (division)
 - **%**: Kalan ya da mod (remainder)

```
int i = 5;  
int j = 7;  
int k = i + j;  
int u = i * j;
```

İkili Aritmetik İşlemciler - II



- Java'da atama ya da kıyaslama vb. bir işlemci olmadan, sadece aritmetik işlem yapılamaz, derleyici **Not a statement** hatası verir

```
int i = 5;  
int j = 7;  
i + j; // "Not a statement" hatası  
int u = i + j; //OK  
int u = 2 * (i + j); //OK
```

Bileşik Atama (Compound Assignment)



- Bir değişkenin değeri değiştirilip yine kendisine atanacaksa, bu **bileşik atama (compound assignment)** ile daha az kod yazarak yapılabilir
- Birleşik atamalar, +, -, *, /, %, &, |, ^, <<, >> ve >>> işlemcilerinde kullanılabilir
- Yazımda önce işlemci sonra = gelir, += gibi
- Dolayısıyla = ile birlikte toplam 12 tane atama operatörü vardır

```
int i = 5;

i = i + 3; // i şimdi 8
i += 3;    // i şimdi 11
i -= 4;    // i şimdi 7
i *= 3;    // i şimdi 21
i /= 7;    // i şimdi 3
i %= 2;    // i şimdi 1
```

AssignmentOperator



- AssignmentOperator

Aritmetik Yükseltme (Promotion) - I



- İkili aritmetik işlemciler, yaptıkları işlem sonucunda en az `int` tipinde bir değer üretirler, yani
 - En az bir işlenen `double` ise sonuç `double`,
 - Değilse, en az bir işlenen `float` ise sonuç `float`,
 - Değilse, en az bir işlenen `long` ise sonuç `long`,
 - Aksi taktirde sonuç `int` tipinde olur

Aritmetik Yükseltme (Promotion) - II



- Yani `int`'den daha küçük tipteki işlenenlerle yapılan aritmetik işlemler, daima `int` bir sonuç üretir
- Buna **yükseltme (promotion)** denir
- İşlenenlerin tipine dönmek için çevrim (cast) gerekir

```
int i = 5; double d = 2.3d;
i = i + d;           // Hata! Toplam double'dır
i = (int) (i + d);    // Doğru! Çevrim (cast) gerekli

byte b1 = 8, b2 = 5;
byte b3 = b1 + b2;    // Hata! Toplam int'dir
byte b3 = (byte) (b1 + b2); // Doğru! Çevrim (cast) gerekli
```

Bileşik Atama ve Yükseltme



- Bileşik atama kullanılması durumunda bir daraltan çevrime (cast) ihtiyaç olursa bu otomatik olarak yapılır
- Otomatik çevrim (cast), bileşik atama kullanılmazsa, yapılmaz

```
int i = 5; double d = 2.3d;
i = i + d;           // Hata!
i = (int) (i + d);   // Çevrim (cast) gerekli

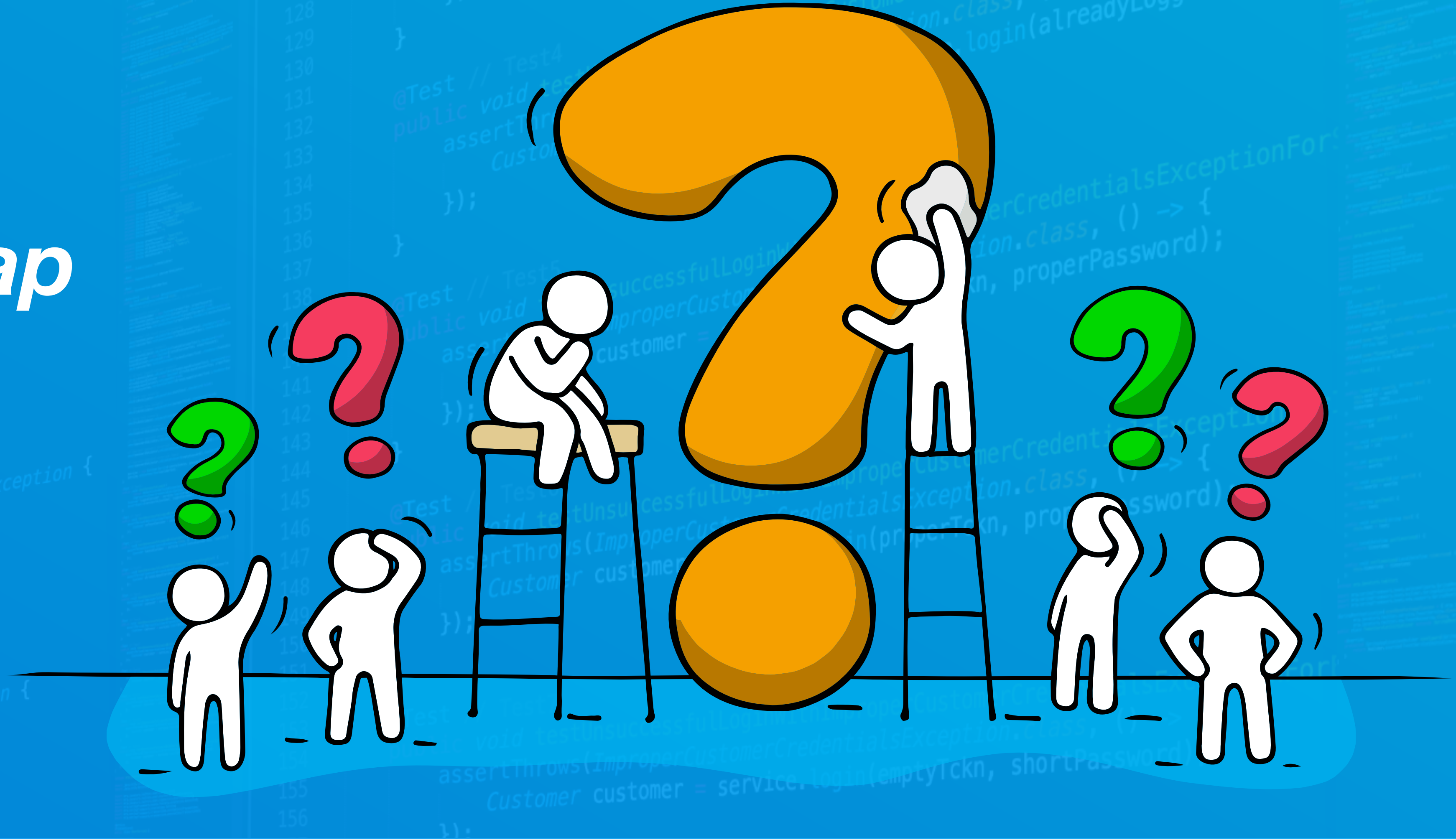
i = 5;
i += d; // i şimdi 7, otomatik çevirim.
i -= d; // i şimdi 4,
i *= d; // i şimdi 9
i /= d; // i şimdi 3
```

ArithmeticPromotion



- `ArithmeticPromotion`

Soru ve Cevap Zamanı!





Tekli Aritmetik İşlemciler

Tekli Aritmetik İşlemciler



- **Tekli aritmetik işlemciler (unary arithmetic operators)** 4 tanedir:
 - **+**: İşleneni etkilemez, negatif değerli değişkenin değerini pozitif yapmaz,
 - Bu sebeple pek kullanılmaz
 - **-**: İşleneni negatif yapar
 - **++**: Artım (increment) işlemcisi, sayısal değeri 1 arttırır
 - **--**: Eksiltme (decrement) işlemcisi, sayısal değeri 1 eksiltir

+ ve - İşlemcileri - I



- **+** ve **-**, bir atama vb. işlemciyle birlikte kullanılmalıdır, aksi takdirde **Not a statement** hatası oluşur
- **+** ve **-**, ikili aritmetik operatörler gibi en az **int** bir sonuç üretir

```
int i = 5;  
-i; // "Not a statement" hatası  
int j = -i; // j ve i -5 değerinde
```

```
byte b1 = 5;  
byte b2 = -b1; // Hata! Çevrim gerekli  
byte b3 = (byte) -b1;  
byte b4 = (byte) +b1;
```

+ ve - İşlemcileri - II



- Bu sebeple **+**'nın tek etkisi `int`'den küçük olan tipleri `int`'e çevirmesidir

```
byte b1 = 5;  
byte b2 = +b1; // Hata! Çevrim gerekli  
byte b3 = (byte) +b1;
```

UnaryOperators



- `UnaryOperators`
- İşlemcilerin hem kullanımına hem de yükseltmelere dikkat edin

Arttırma ve Eksiltme İşlemcileri - I



- **++** ve **--**, işlenenin öncesinde ya da sonrasında kullanılabilir:
- Öncesinde ise, **önce arttırma (pre-increment)** ya da **eksiltme (pre-decrement)** yapılır, sonra işlenenin değeri belirlenir,
- Sonrasında ise, önce işlenenin değeri belirlenir varsa atama yapılır, **sonra arttırma (post-increment)** ya da **eksiltme (post-decrement)** yapılır

```
int i = 5;
int j = ++i; // i ve j şimdi 6
j = i++;     // j halen 6 ama i 7 oldu
j = --i;     // i ve j şimdi 6;
j = i--;     // j şimdi de 6 ama i 5 oldu
```

Arttırma ve Eksiltme İşlemcileri - II



- **++** ve **--** işlenenin tipini korur, yükseltme yapmaz

```
byte b1 = 5;  
byte b2 = -b1;      // Hata! Çevrim gerekli  
b2 = (byte) -b1;  
byte b3 = b1++;     // Çevrim gerekli değil
```

- **++** ve **--** atama olmadan tek başına kullanılabilir

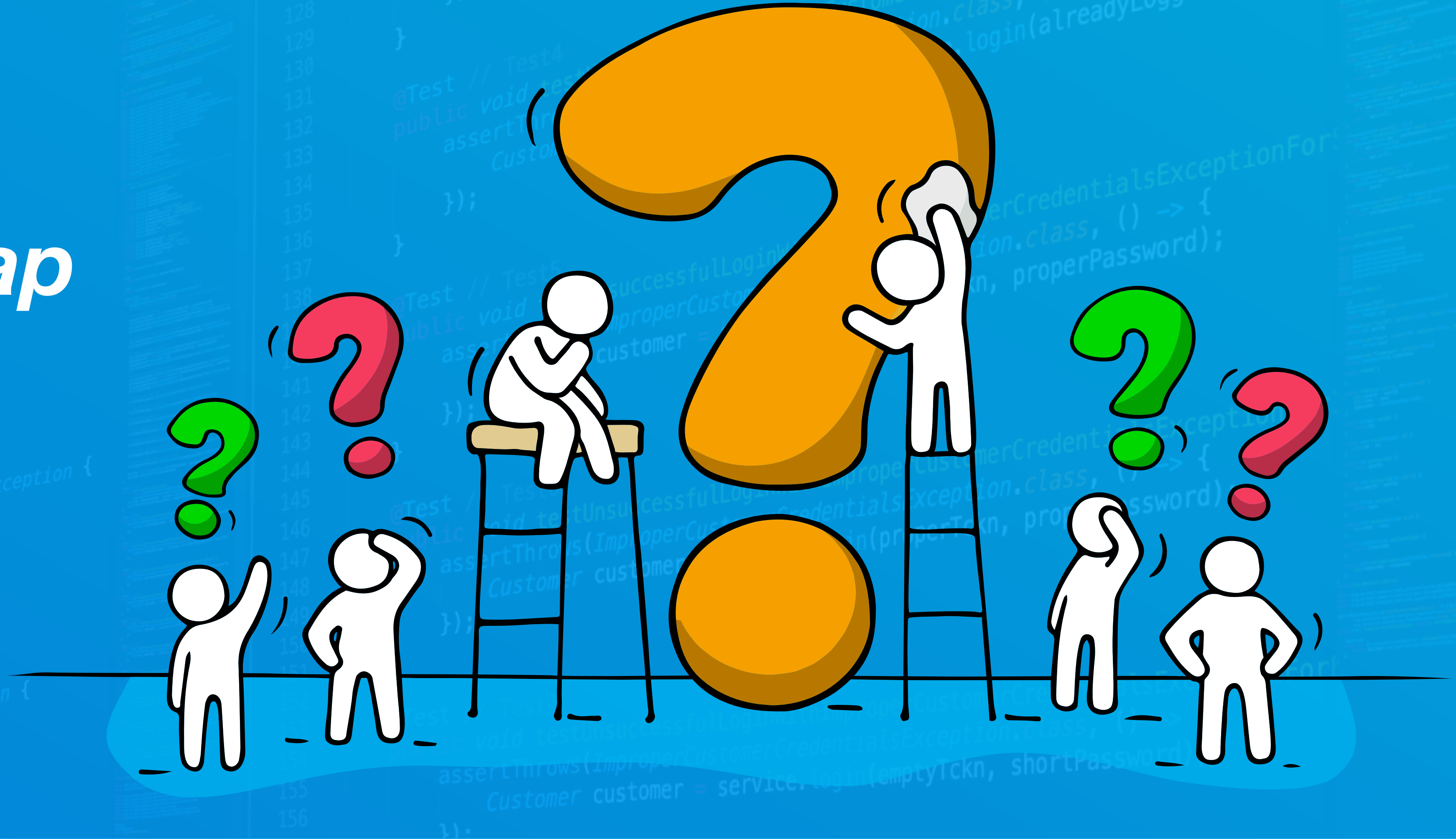
```
int i = 5;  
++i;    // i 6 oldu  
i++;    // i 7 oldu  
--i;    // i 6 oldu  
i--;    // i 5 oldu
```

UnaryIncAndDecOperators



- `UnaryIncAndDecOperators`
- Bu örnekteki `weirdBehavior` metodunda olup biteni açıklamaya çalışın

Soru ve Cevap Zamanı!



İlişkisel (Kıyas) İşlemcileri



Kıyaslama İşlemcileri - I



- **İlişkisel ya da kıyaslama işlemcileri (relational , comparison operators)**, 6 tanedir ve iki işlenen alıp, onlar arasındaki ilişkiyi sorgular, onları kıyaslar ve bir mantıksal değer üretir
- 4 tane büyüklük kıyaslama işlemcisi, iki sayısal (tam ya da gerçekte sayı) işlenen alır ve **boolean** bir sonuç üretir:
 - **>** Büyük müdür? **>=** Büyük ya da eşit midir?
 - **<** Küçük müdür? **<=** Küçük ya da eşit midir?

Kıyaslama İşlemcileri - II



- Eşitlik kıyaslayan işlemciler ise iki tane sayısal, `boolean` ya da referans işlenen alır ve `boolean` bir sonuç üretirler:
 - `==` Eşit midir?
 - `!=` Eşit değil midir?
- `char` işlenenler ile kullanıldığında kıyaslama işlemcileri, işlenenlerin tamsayı değerlerini kıyaslar

== İşlemcisi



- Eşitlik kıyaslamak için **==** kullanın
- **if** ya da **while** gibi yapılarda yanlışlıkla eşitlik kıyaslaması için **==** yerine atama işlemcisi **=** yazmak, yaygın bir hatadır

```
int i = 5, j = 10;  
boolean b = i == j; // false  
boolean b = i = j; // Error!
```

```
int i = 5, j = 10;  
if(i == j) {...} // false  
if(i = j) {...} // Error!
```

- **==**, işlenenlerin değerlerinin eşit olup olmadıklarını kıyaslar:
- Bu yüzden **==** operatörü ile basit tiplerin değerleri referansların ise adresleri kıyaslanır

Referanslarda == İşlemcisi - I



- **==** işlemcisi referanslara uygulandığında, adresleri kıyaslandığından, ancak ve ancak iki referans da aynı adresi yani aynı nesneyi gösteriyorsa **true** döndürür
- **==** işlemcisi, referansların gösterdiği nesnelerin içeriklerinin yani durumlarının aynı olup olmadığını kontrol etmez
- Bunu yapmanın yolu farklıdır, ileride ele alınacaktır

Referanslarda == İşlemcisi - II



```
String s1 = new String("String");
String s2 = new String("String");
if (s1 == s2)
    System.out.println("The same");
else
    System.out.println("Different"); // Different

s1 = s2;    // Equalizing the references

if (s1 == s2)
    System.out.println("The same"); // The same
else
    System.out.println("Different");
```

Referansların null Kontrolü



- Referansların `null` olup olmadığını kontrol etmek için `==` ve `!=` işlemcileri kullanılır
- Eğer referansın gösterdiği hiçbir nesne yoksa `== true`, `!= false` döndürür

```
Car car = getMeCar();
if(car != null)
    car.go(100);
else
    System.out.println("Ref is null!");
```

RelationalOperators



- RelationalOperators

Aralık Kıyaslaması



- Java'da **aralık kıyaslaması** (**range comparison**) ancak şart işlemcileri ile yapılabilir
- Yani şu kod hatalıdır: `boolean b = 3 < i < 7; // Derleme hatası`
- Bunun yerine şart işlemcileri kullanılarak çoklu kıyaslama yapılmalıdır

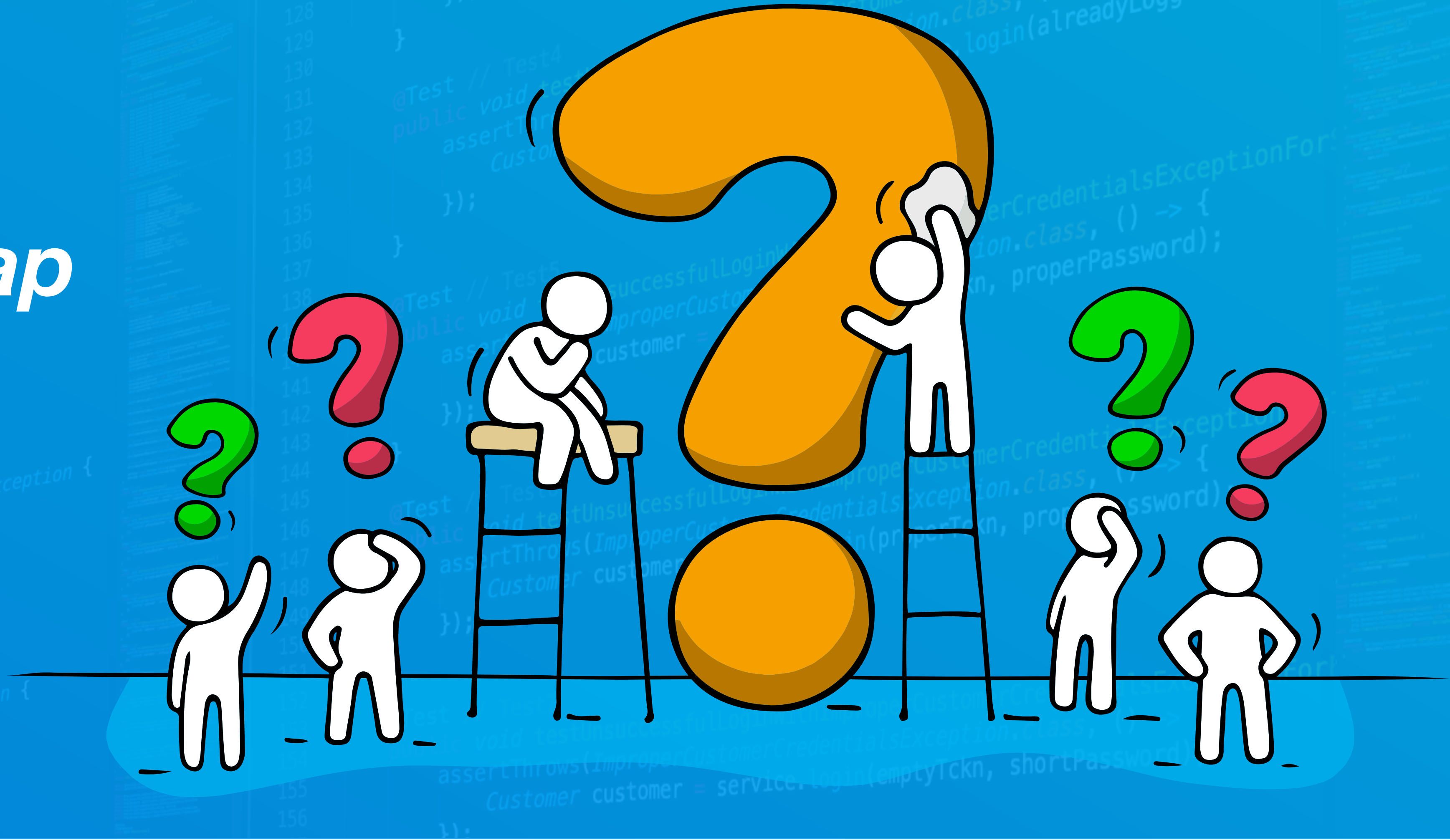
```
boolean b = (3 < i) && (i < 7);
```

RangeComparison



- RangeComparison

Soru ve Cevap Zamanı!



Mantıksal işlemler



Mantıksal İşlemciler



- **Mantıksal (logical)** işlemciler 6 tanedir ve bir ya da iki mantıksal (**boolean**) işlenenler alır ve **boolean** bir değer üretir
- **&** (ampersand) **VE (AND)**, ikili işlemcidir,
- **|** (vertical bar, pipe) **VEYA (OR)**, ikili işlemcidir,
- **^** (caret) **DIŞLAYICI (XOR)**, ikili işlemcidir,
- **!** (exclamation mark) **DEĞİL (NOT)**, tekli işlemcidir

Kısa Yol & ve | - |



- **&** ve **|**'in kısa yolları (short circuit) vardır:
 - **&&**: Kısa yol **&**'dir ve sağ taraf, ancak sol taraf doğruysa (**true**) çalışır
 - **||**: Kısa yol **|**'dir ve sağ taraf, ancak sol taraf yanlışsa (**false**) çalışır
- Bu sebeple kısa yollar daha etkin çalışma zamanı sağlar
- Kısa yollarda, sağ taraflarındaki ifadelerde metot çağırısı, atama vs. varsa, bunun yapılmayabileceğine dikkat edin

Kısa Yol & ve |



- **&&** ve **||** 'de sağ taraf, sol tarafa bağlı olarak değerlendirildiğinden bu iki işlemciye **şartlı VE** ve **şartlı OR** (**conditional-AND** ve **conditional-OR**) işlemcisi de denir

LogicalOperators



- LogicalOperators

Tekli Mantıksal İşlemci - I



- Tekli mantıksal işlemci (logical unary operator) bir tanedir: **!**
- **!**, mantıksal değer **tersini** ya da **değilini** (logical complement) alır
- Dolayısıyla **true** bir değerden **false**, **false** bir değerden **true** üretir

```
boolean b1 = true;  
boolean b2 = !b1; // b2 is false  
boolean b3 = !false; // b3 is true
```

- **!** bir atama vb. bir işlemciyle birlikte kullanılmalıdır, aksi takdirde derleme hatası oluşur

```
boolean b1 = true;  
!b1; // Derleme hatası
```

Tekli Mantıksal İşlemci - II



- **!=** şeklinde bir birleşik atama (compound assignment) yoktur

```
boolean b1 = true;  
b1 != b1; // Derleme hatası
```

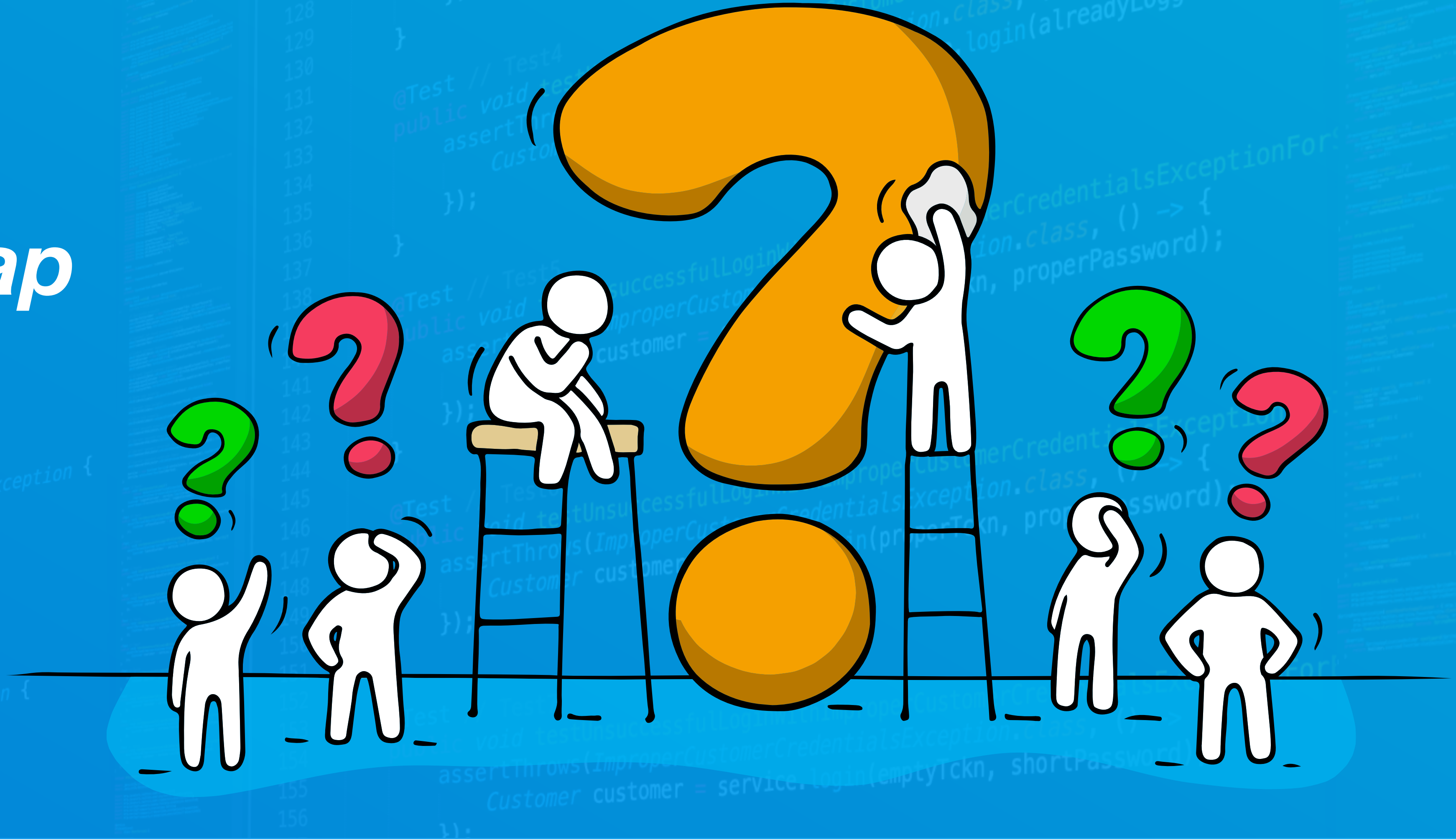
- **!=** kıyaslama yapan, eşitsizliği yani eşit olmamayı değerlendiren bir işlemcidir
- Daha önce ele alınmıştı

UnaryLogicalOperator



- `UnaryLogicalOperator`

Soru ve Cevap Zamanı!



Şartlı işlemler



Üçlü Şart İşlemcisi - I



- Üçlü şart işlemcisi (ternary conditional operator), **?:** 3 tane işlenen alan tek işlemcidir
- İlk ifade **boolean** olmalıdır ve doğru ise ikinci, değil ise üçüncü ifade değer olarak üretilir
- Sıklıkla üretilen değer bir değişkene atanır

operand1 ? operand2 : operand3

```
int i = 8; int j = 11;  
int min = (i ≤ j) ? i : j;
```

Üçlü Şart İşlemcisi - II



- İlk ifade `boolean` olmalıdır ve doğru ise ikinci ifade, değil ise üçüncü ifade değerlendirilir ve varsa en soldaki değere atanır
- İkinci ya da üçüncü ifade `void` döndürmeyen bir metot da olabilir
- İkinci ve üçüncü ifadelerden sadece birisi değerlendirilir, ikinci ifade değerlendirilirse üçüncüsü, benzer şekilde üçüncü ifade değerlendirilirse de ikincisi değerlendirilmez
- Bu sebeple bu iki ifadeden birisinin çalıştırılmayacağını göz önünde bulundurmak gerekir

Üçlü Şart İşlemcisi - III



- Üçlü şart işlemcisinin sonucunun bir değişkene atanması gerekmez; üretilen değer bir metota argüman olarak geçilebileceği gibi metottan geriye de döndürülebilir

```
public static void withMethods() {  
    ...  
    int i = 73;  
    int j = 4;  
    int seconds = 8;  
    boolean isEven = isEven(seconds % 2 == 0 ? i : j);  
}  
  
public static boolean isEven(int arg) {  
    return arg % 2 == 0; // Is it even?  
}
```

Üçlü Şart İşlemcisi - IV



- Üçlü şart işlemcisi, ileride ele alınacak olan `if-else` cümlesinin kısa halidir
- Bu sebeple sık tercih edilmez
 - Pek çok geliştirici, `if-else` yapısını kullanmayı, okunurluk açısından ve çoğunlukla da alışkanlıkla tercih eder
- Üçlü şart işlemcisinin iç-içe kullanımı söz konusuysa, bu durumda `if-else` yapısı, okunurluktan dolayı tercih edilmelidir

TernaryOperator



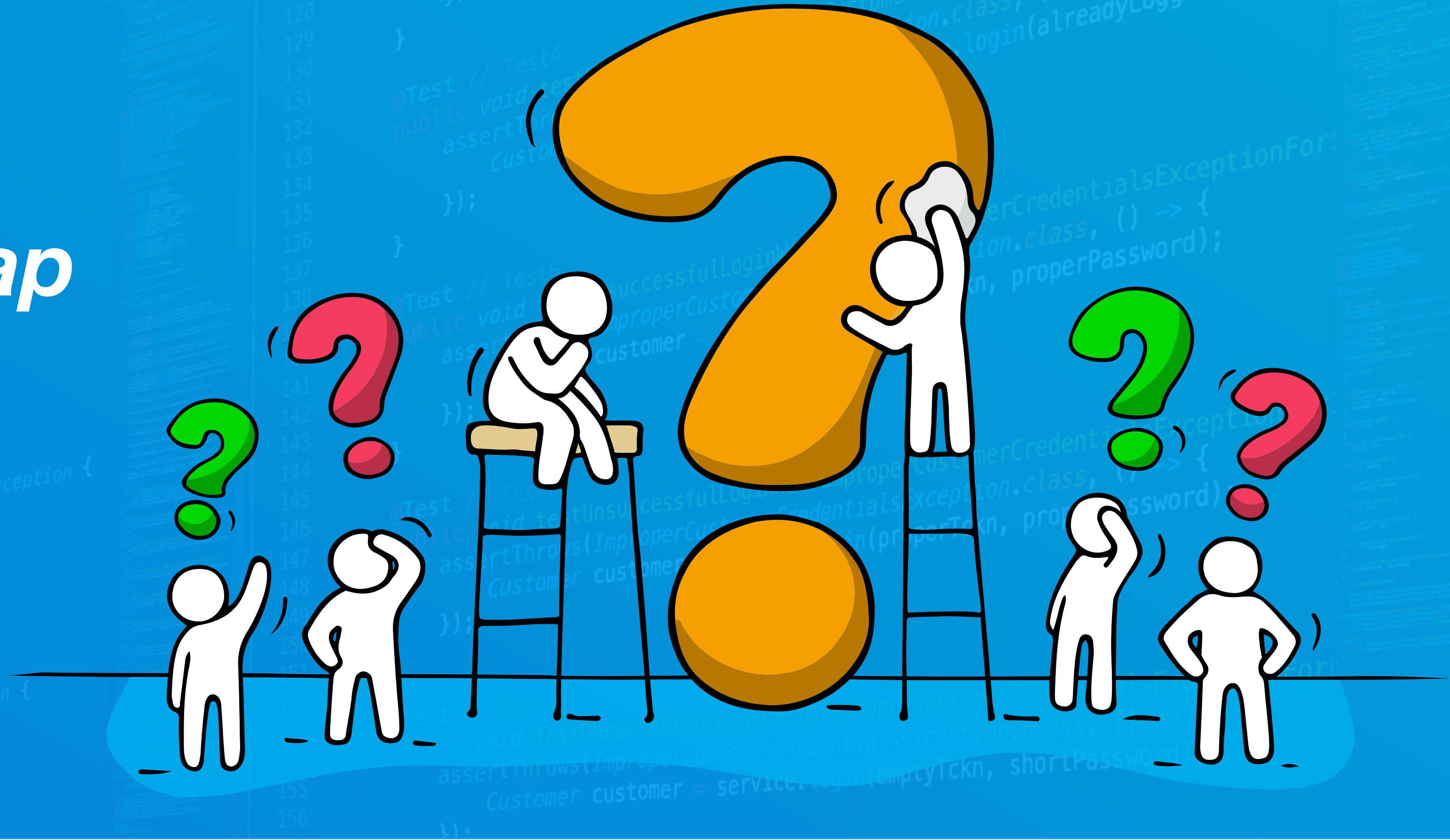
- TernaryOperator

Şartlı İşlemciler: Kısa Yol & ve |



- Daha önce bahsetmiştik:
 - **&&** ve **| |** 'de sağ taraf, sol tarafa bağlı olarak değerlendirildiğinden bu iki işlemciye **şartlı VE** ve **şartlı OR** (**conditional-AND** ve **conditional-OR**) işlemcisi de denir

Soru ve Cevap Zamanı!



Bit işlemcileri





- **Bit işlemcileri (bitwise operators)**, tam sayı işlenenlerin bit yapıları, bit desenleri üzerinde çalışır
- Bit seviyesinde işlem yapan iki işlemci grubu vardır:
 - Mantıksal (logical)
 - Kaydırma (shift)
- Bit işlemcileri daha çok, gömülü (embedded) vb. uygulamalarda kullanılır



Mantıksal Bit İşlemcileri

Mantıksal Bit İşlemcileri - I



- Mantıksal bit işlemcileri (bitwise logical operators):
 - **&**: İki işlenenin bitlerini **VE (AND)** ile işler,
 - **|**: İki işlenenin bitlerini **VEYA (OR)** ile işler,
 - **^**: İki işlenenin bitlerini **DIŞLAYICI (EX-OR)** ile işler,
 - **~**: (tilde) Bir işlenenin bitlerinin **TÜMLEYENİ (COMPLEMENT)** alır

```
int i = 7;    // 0111
int j = 4;    // 0100
int k = i & j; // 0100 → 4
```

```
int i = 7;    // 0111
int j = 4;    // 0100
int k = i | j; // 0111 → 7
```

```
int i = 7;    // 0111
int j = 4;    // 0100
int k = i ^ j; // 0011 → 3
```

```
int i = 7;    // 00...0111
int j = ~i;    // 11...1000 → -8
```

Mantıksal Bit İşlemcileri - II



- Bu işlemciler tam sayı tipleri üzerinde işlem yaparlar ve gerekirse yükselterek en az bir `int` üretirler

```
byte i = 7;    // 0111
byte j = 4;    // 0100
byte k = i & j; // Error!
byte k = (byte) (i & j); // OK
```

Mantıksal Bit İşlemcileri - II



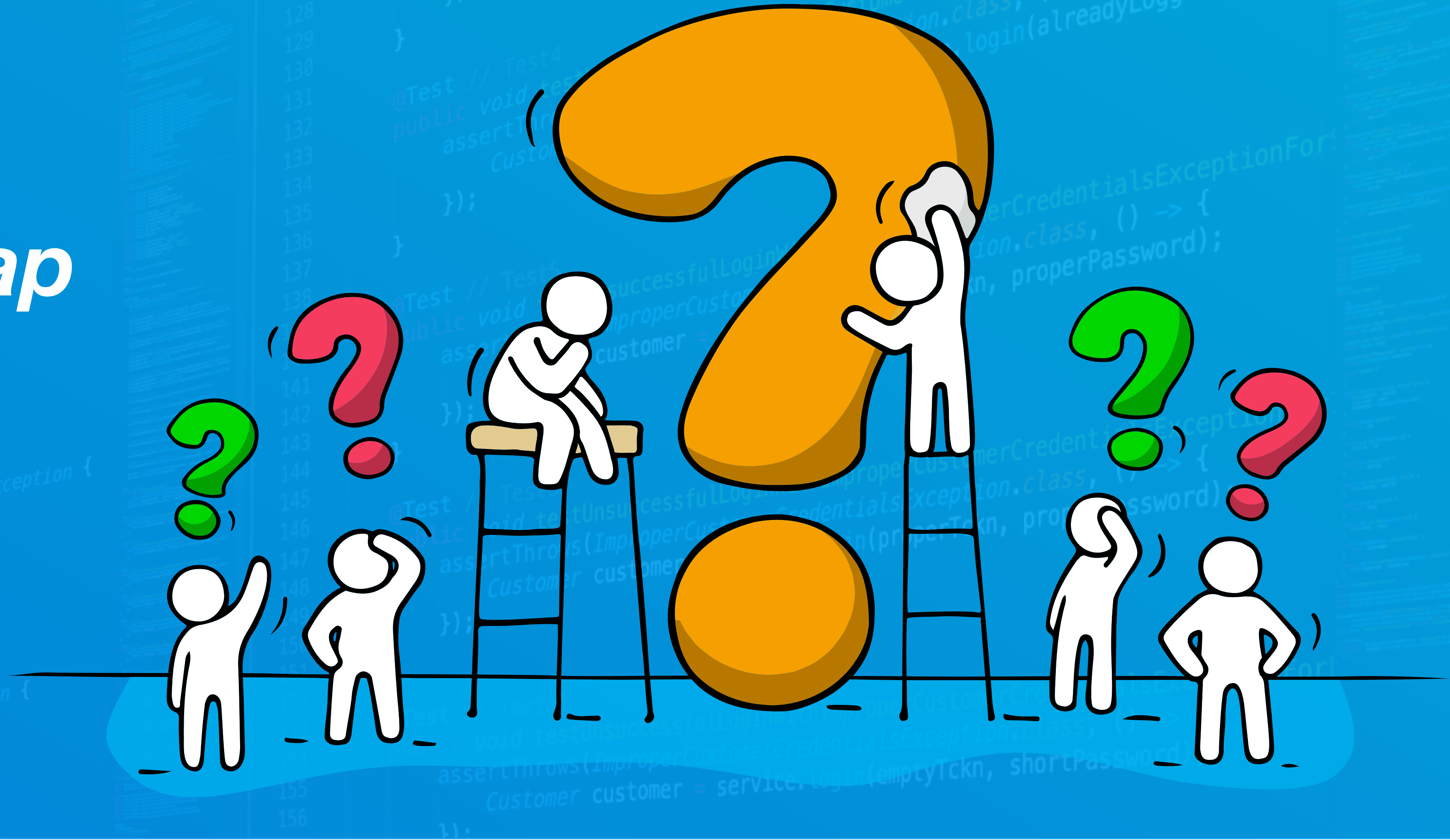
- Mantıksal bit işlemcileri ile mantıksal işlemcilerin iki tanesi (& ve |) sembol olarak aynıdır
- Dolayısıyla & ve | çoklu kullanılmış (overloaded) işlemcilerdendir
- İşlenenler `boolean` ise işlemci mantıksal işlem yapar ve sonuç `boolean` olur, işlenenler tam sayı ise işlemci bit işlem yapar ve sonuç tam sayı olur
- Bu işlemcilerde kısa yol tanımlı değildir

BitwiseLogicalOperators



- `BitwiseLogicalOperators`
- Bu örnekteki tam sayıların ikili karşılıklarına (binary representation) bakarken Java'nın **ikinin tümleyeni (2's complement)** yaklaşımını kullandığını unutmayın

Soru ve Cevap Zamanı!





Kayıtma İşlemcileri

Kaydırma İşlemcileri - I



- **Kaydırma işlemcileri (shift operators)** işlenenin bit dizisini, verilen kadar sağa ya da sola kaydırırlar
- İki tam sayı işlenen alırlar ve en az bir `int` üretirler
- **>>**: `x`'i `y` kadar sağa kaydırma `x >> y`
- **<<**: `x`'i `y` kadar sola kaydırma `x << y`
- **>>>**: `x`'i `y` kadar sağa, işaretsiz kaydırma `x >>> y`

Kaydırma İşlemcileri - II



- Sağa kaydırma 2'ye bölme, sola kaydırma 2 ile çarpmaktır
- Tek sayı halinde ikiye bölme tam sayı üretecektir
- **<<**'da en sağa 0 eklenir
- **>>**'da en sola, sayının işaretine göre 0 ya da 1 eklenir

```
int i = 1; // 0000 0000 0000 0001 1
i = i << 7; // 0000 0000 1000 0000 128
i = i >> 7; // 0000 0000 0000 0001 1
i = i >> 1; // 0000 0000 0000 0000 0

i = -1; // 1111 1111 1111 1111 -1
i = i << 1; // 1111 1111 1111 1110 -2
i = i << 6; // 1111 1111 1000 0000 -128
i = i >> 2; // 1111 1111 1110 0000 -32
```

```
// i is odd
int i = 139; // 0000 0000 1000 1011 139
i = i >> 1; // 0000 0000 0100 0101 69
i = i >> 1; // 0000 0000 0010 0010 34
i = i >> 1; // 0000 0000 0001 0001 17
i = i >> 1; // 0000 0000 0000 1000 8
```

Kaydırma İşlemcileri - II



- **>>>**'da en sola daima 0 eklenir çünkü işaret pozitif olur

```
int i = 64;  
// 0000 0000 0000 0000 0000 0000 0100 0000 64  
i = i >>> 5;  
// 0000 0000 0000 0000 0000 0000 0000 0010 2  
  
int i = -64;  
// 1111 1111 1111 1111 1111 1111 1100 0000 -64  
i = i >>> 5;  
// 0000 0111 1111 1111 1111 1111 1111 1110 134217726
```

- Sağa işaretsiz kaydırma genelde sayılar için yapılmaz
- Sola işaretsiz kaydırma için **<<<** gibi bir işlemci yoktur

Kayıdırma İşlemcileri - IV



- Sağa işaretli kaydırma genelde sayılar için yapılmaz
- Örneğin görüntü işlemede (image processing) renkler çoğunlukla *AARRGGBB* formatında `int` değerler olarak saklanır
- Bu formatta en soldaki 8 bit alpha kanalı değeridir

```
int pixel = 0xFF123456; // A very opaque pixel
// 11111111000100100011010001010110

// Extracting Alpha channel (the top 8 bits)
int alpha = pixel >>> 24; // result is 255 (0xFF, 11111111)
```

Kaydırma İşlemcileri - V



- Kaydırma işlemlerinde kaydırma miktarı, bit dizisi kaydırılacak olan sayı `int` ise mod 32, `long` ise mod 64 ile hesaplanır

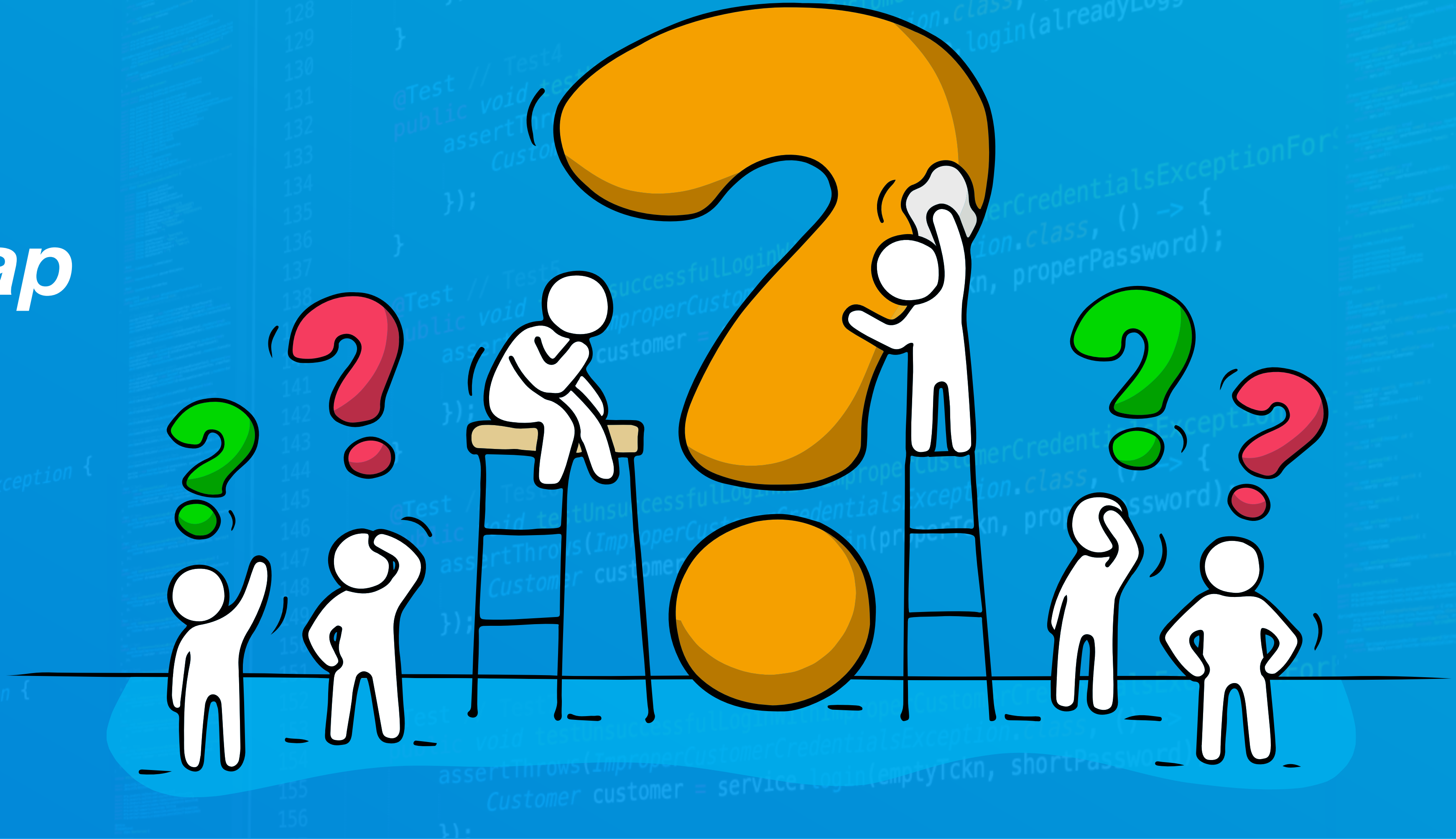
```
int i = 64;  
// 0000 0000 0000 0000 0000 0000 0100 0000 64  
  
i = i >> 5;  
// 0000 0000 0000 0000 0000 0000 0000 0010 2  
  
i = 64;  
i = i >> 37;  
// 0000 0000 0000 0000 0000 0000 0000 0010 2
```

ShiftOperators



- `ShiftOperators`

Soru ve Cevap Zamanı!



Diğer işlemler



Diğer İşlemciler



- Bunlar dışında başka işlemciler de vardır örneğin `instanceof` veya `->`
- Bunlar ileriki eğitimlerde ele alınacaktır



Not



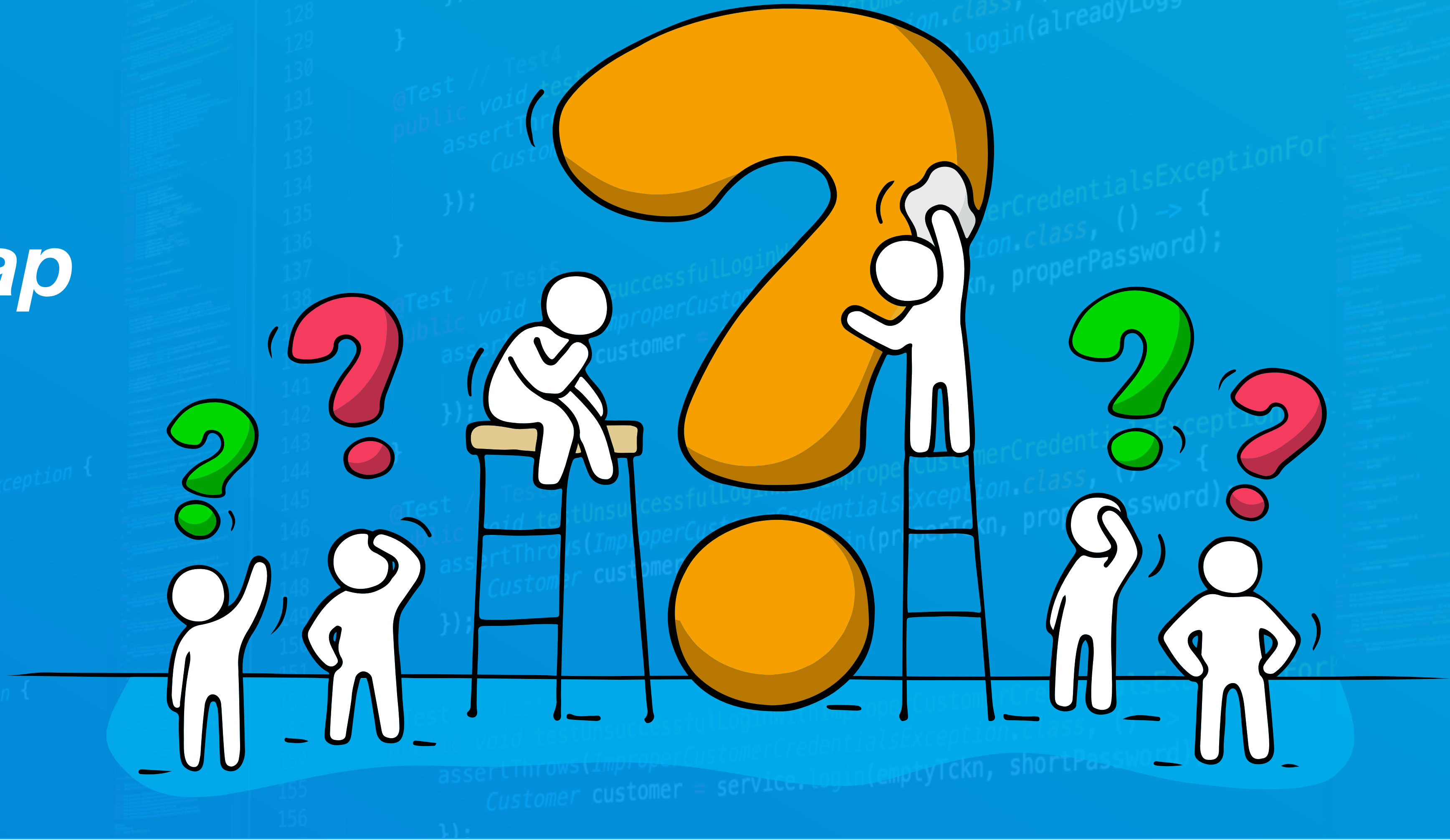
- Visual Basic, Fortran, JavaScript ya da Python gibi dillerde olan ve $\wedge$ veya `**` ile temsil edilen üst alma işlemcisi (power ya da exponentiation operator) Java'da yoktur
- Bu amaçla `java.lang.Math` sınıfında `pow()` metodu vardır

```
double pow(double a, double b)
```

```
double squareRootOfTwo = Math.pow(2, 0.5);
```

- İleride ele alınacaktır

Soru ve Cevap Zamanı!



Öncelik ve Değerlendirme Sırası



Öncelik ve Değerlendirme Sırası - I



- Birden fazla işlemci aynı cümlede kullanılırsa hangisinin önce çalışacağı, **öncelik sırası (precedence)** ile, işlemcinin birden fazla işleneni varsa, hangisini önce işleyeceği ise **değerlendirme sırası (evaluation order ya da associativity)** ile belirlenir

```
2 + 8 * 5 ⇒ 2 + (8 * 5) // Precedence
```

```
a = b = c = 4 ⇒ a = (b = (c = 4)) // Associativity
```

Öncelik ve Değerlendirme Sırası - II



Level	Category	Operators	Associativity
1	Postfix	expr++, expr--	Left-to-Right
2	Unary	++expr, --expr, +, -, ~, !	Right-to-Left
3	Multiplicative	*, /, %	Left-to-Right
4	Additive	+, -	Left-to-Right
5	Shift	<<, >>, >>>	Left-to-Right
6	Relational	<, >, <=, >=, instanceof	Left-to-Right
7	Equality	==, !=	Left-to-Right
8	Bitwise AND	&	Left-to-Right
9	Bitwise XOR	^	Left-to-Right
10	Bitwise OR		Left-to-Right
11	Logical AND	&&	Left-to-Right
12	Logical OR		Left-to-Right
13	Ternary	? :	Right-to-Left
14	Assignment	=, +=, -=, *=, /=, %=, &=, ^=, =, <<=, >>=, >>>=	Right-to-Left

Öncelik - I



- Bir işlemci değerlendirilmeden önce işlenenleri değerlendirilir,
 - Bunun istisnası `&&`, `||` ve `?:` işlemcilerdir
- Çünkü bu işlemcilerin karar vermesi için tüm işlenenlerini değerlendirmesi gerekmez
 - `&&`, `||` kısa yol işlemcileridir, sağ solun değerine göre değerlendirir
 - `?:` ise iki şeyden birine karar verir, karar verdiğini değerlendirir

Öncelik - II



- Öncelik ile ilgili olarak şu kurallar vardır:
 - Öncelik sıralamasında daha yukarıda olan işlemciler daha aşağıda olanlardan önce değerlendirilirler
 - Aynı önceliğe sahip işlemciler bir arada bulunuyorlarsa, değerlendirme sırası soldan sağa olur
 - Parantezler değerlendirmede önceliğe sahiptirler
 - Birden fazla parantez varsa parantezler soldan sağa değerlendirilir

Öncelik - III



- Metot çağrısında parametreler, soldan sağa değerlendirilir

Öncelik - IV



- Öncelik sırasını ezbere bilmek gerekli değildir, bu konudaki ilkelerin bilinmesi önemlidir
- Ayrıca öncelik sırasını bilmeyi gerektiren kod yazmak ise iyi bir uygulama değildir
- Aslolan, gerektiğinde daha fazla satır ve parantez kullanarak okunabilir kod yazmaktır
- Anlaşılması, önceliklendirme sırasını bilmeye bağlı olan kod yazmak kötü bir alışkanlıktır, kaçınılmalıdır

Öncelik - V



- Örneğin aşağıdaki cümlelerde doğru sonucu bulmak, sonucun kullanılan işlemcilerin önceliğine bağlı olmasından dolayı kolay değildir

```
int a = 6;
int b = 5;
int c = 10;
float result = a + ++b * c / a * b; // result?

boolean b1 = false, b2 = true, b3 = true;
boolean bool = b1 & b2 | b3; // bool?
```

Değerlendirme Sırası - I



- **Değerlendirme sırası (evaluation order, associativity)**, bir işlemcinin, işlenenlerini hangi sırayla işlediğidir
- Sonra gelen arttırma ve eksiltme işlemcileri dışında bütün tekli işlemciler, sağdan sola doğru işlerler
- Bütün ikili işlemciler soldan sağa doğru işlerler,
- Bunun tek istisnası ise atama (=) işlemcisidir
- Birleşik atama işlemcileri de bu şekilde davranır

Değerlendirme Sırası - II



```
int i = 8;    // From right to left
i = i++;     // From right to left
i = ++i;     // From left to right

a = b = c = 4 ⇒ a = (b = (c = 4))

// From left to right: The same meaning
a + b + c    and (a + b) + c
a - b - c    and (a - b) - c

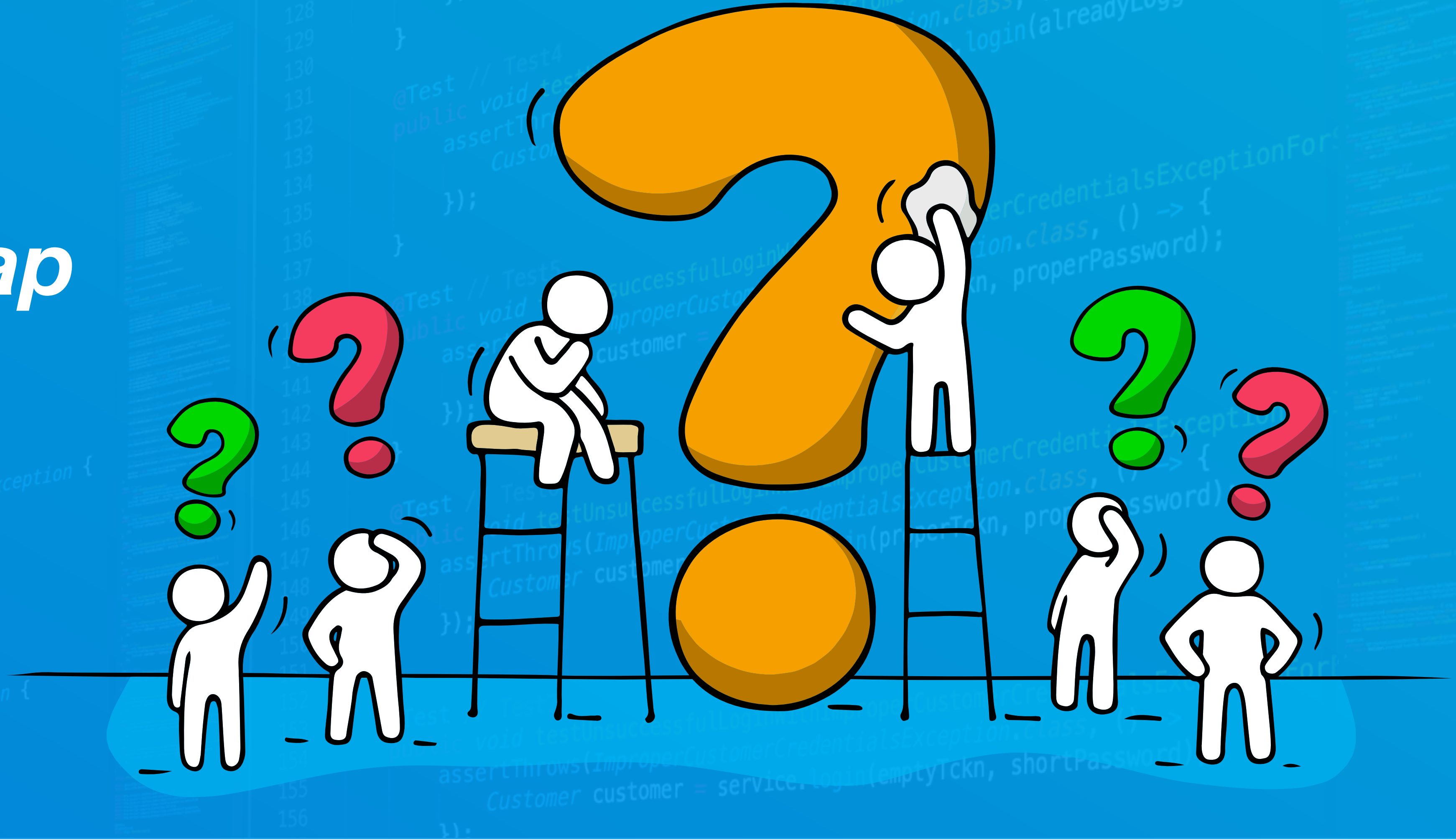
// From left to right
new Student().register();
(new Student()).register()
```

OperatorPrecedence



- OperatorPrecedence

Soru ve Cevap Zamanı!



Sayısal Yükselelmeler



Sayısal Yükseltme (Numeric Promotion)



- **Sayısal yükseltmenin (numeric promotion)**, belli işlemcilerde, işlenenlerin tiplerini, en az bir `int` sonuç üretecek şekilde yükselttiğini daha önce belirtmiştik
- Aritmetik yükseltmelerin iki türü vardır:
 - Tekli işlemciler yükseltme
 - İkili işlemcilerde yükseltme

Tekli Yükseltme - I



- Eğer tekli işlemcinin işleneni `int`'den daha küçük bir tip ise, işlenenin tipi `int`'e yükseltilir, aksi takdirde yükseltme yapılmaz
- Tekli yükseltme şu işlemcilerde yapılır:
 - `+`, `-`, `~`
 - `>>`, `<<`, `>>>`: Bu işlemcilerde yükseltme ayrı ayrı yapılır, yani kayacak sayı `long` ise mesafe de `long` olmaz, sadece `int`'e yükseltilir

Tekli Yükseltme - II



- Dizi indekslerinde (array index) içinde ve dizi büyüklüklerinde (array size), "[]" içinde `int`'e yükseltme yapılır
- Diğer tür işlemcilerde, örneğin `++` ve `--`'de, yükseltme yapılmaz

UnaryPromotion



- `UnaryPromotion`

İkili Yükseltme - I



- İki işlenen alan işlemcilerde, işlenenlerden birisinin tipi `double` ise diğeri de `double`, değilse, birisinin tipi `float` ise diğeri de `float`, değilse, birisinin tipi `long` ise diğeri de `long`, değilse ikisi de gerekiyorsa `int` tipine yükseltilir
- Şu ikili işlemcilerde yükseltme yapılır:
 - Aritmetik işlemcilerde: `+` `-` `*` `/` `%`
 - Kıyaslama işlemcilerinde: `<` `<=` `>` `>=` `==` `!=`

İkili Yükseltme - II



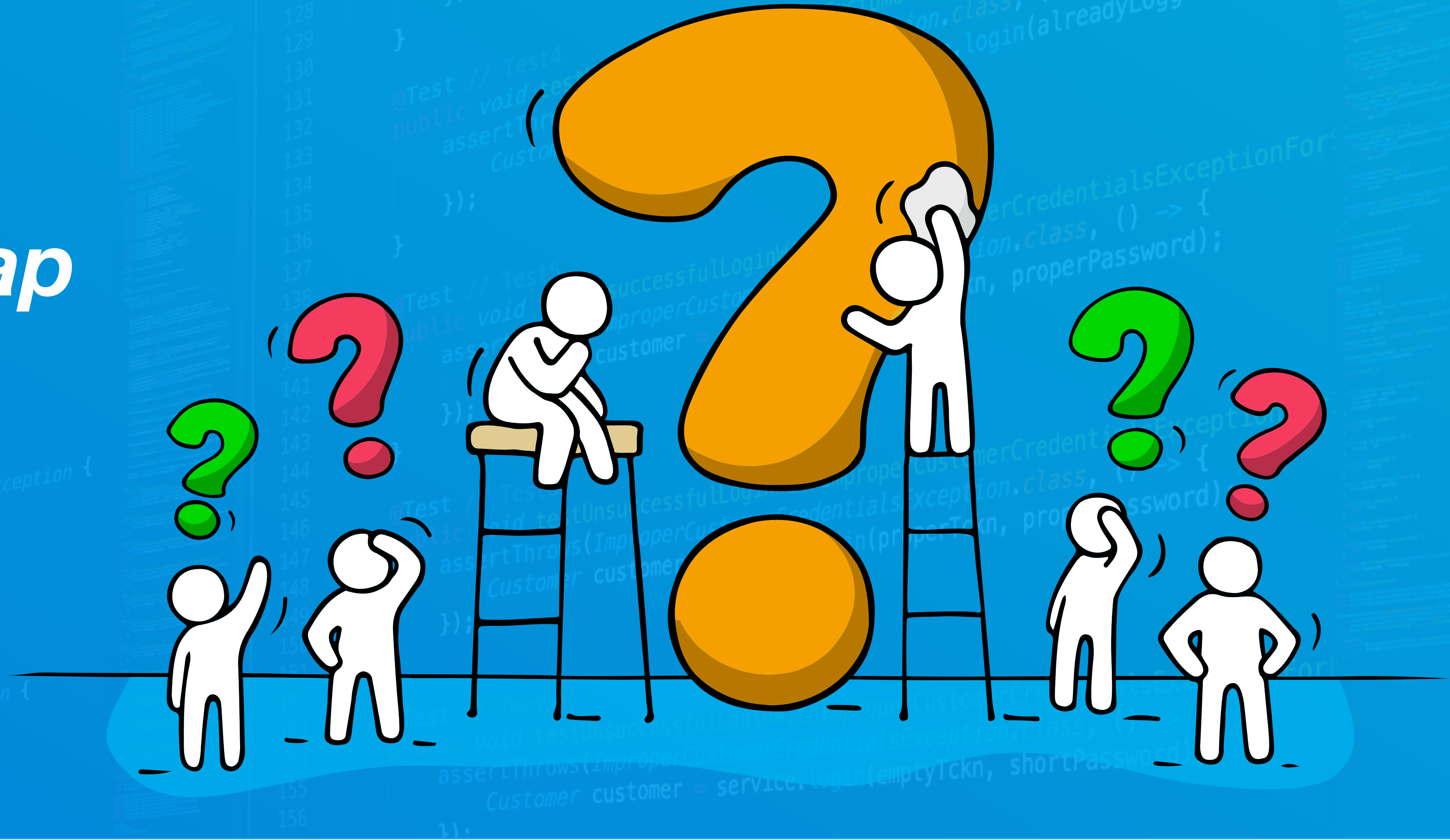
- Bit işlemcilerinde: $\&$ $|$ $\wedge$
- Bazı durumlarda $?:$ 'de de yükseltme yapılır

BinaryPromotion



- `BinaryPromotion`

Soru ve Cevap Zamanı!



İşlemci Çoklu Kullanımı



İşlemci Çoklu Kullanımı



- Aynı işlemcinin farklı bağlamlarda farklı işlevler yerine getirmesine **işlemci çoklu kullanımı (operator overloading)** denir
- Farklı bağlamdan kasıt genelde farklı işlenenlerdir
- Farklı işlevden kasıt ise tamamen farklı işlerdir
- İşlemcilerin programcılar tarafından tekrar tanımlanabilmeleri, C++ gibi bazı dillerde mümkündür ve bu şekilde aynı işlemci farklı bağlamlarda farklı işlevler görür

Java'da İşlemci Çoklu Kullanımı



- Java işlemcilerin tekrar tanımlanarak çoklu kullanımını sıklıkla uygulayan bir dil değildir
- Hatta işlemcilerin programcılar tarafından tekrar tanımlanmalarına yani overload edilmelerine izin vermez

+ Çoklu Kullanımı



- Java'da birden fazla bağlamda kullanılan dolayısıyla tekrar tanımlanmış çok az işlemci vardır:
 - En bilinen overloaded işlemci +'dır
- + sayılara uygulandığında toplama yapar,
- `String` ile kullanımda, `String` nesnelerini arka arkaya ekler (concatenation)

Diğer Çoklu Kullanılan İşlemciler



- `&` ve `|` de tekrar tanımlanmış işlemcilerdendir
- İşlenenleri **boolean** olduğunda mantıksal (logical) işlemci olur ve **boolean** sonuç üretirken, işlenenleri tam sayı (**byte**, **char**, **short**, **int** ve **long**) olduğunda ise bit seviyesinde (bitwise) işlem yaparlar ve yine bir tamsayı üretirler

```
boolean b = (i < j) & (i > k);
```

```
int k = i & j;
```

OverloadedAndOr



- `OverloadedAndOr`

String Eklemede + - |



- `+`, `String` nesnelerinin art arda birbirlerine eklenmeleri için de kullanılır
- Eğer `+` işlemcisinin işlenenlerinden bir tanesi `String` ise, sonuç `String` nesnesi ile diğer işlenenin `String` formunun arka arkaya eklenmesidir
- Bu amaçla diğer işlenen `String`'e çevrilir

```
"The square root of 2 is " + Math.sqrt(2)
```

```
"The square root of 2 is 1.4142135623730952"
```

String Eklemede + - II



- Unutmayın, + işlemcisi işleme soldan başlar (left-associative)
- Bu yüzden + işlemcisinin toplama mı yapacağı yoksa arda arda ekleme mi yapacağı soldaki işlenenin tipiyle belirlenir

```
"Java" + 1 + 2 // Java12
```

```
1 + 2 + "Java" // 3Java
```

StringConcat



- `StringConcat`

char Eklemede + - |



- `+`, `char` değerlerin art arda birbirlerine eklenmeleri için kullanıldığında, karakterlerin tamsayı değerlerini toplar ve `int` bir sonuç üretir
- Çünkü iki tane karakterin yan yana gelip yeni bir `char` oluşturması mümkün değildir
- Bu durum `+`'ın `String` nesnesi ile kullanımından farklıdır
- Dolayısıyla `+`, `String` nesnelerini ard arda eklerken (concatenation), `char`'ın tam sayı değerleri toplar

char Eklemede + - ||



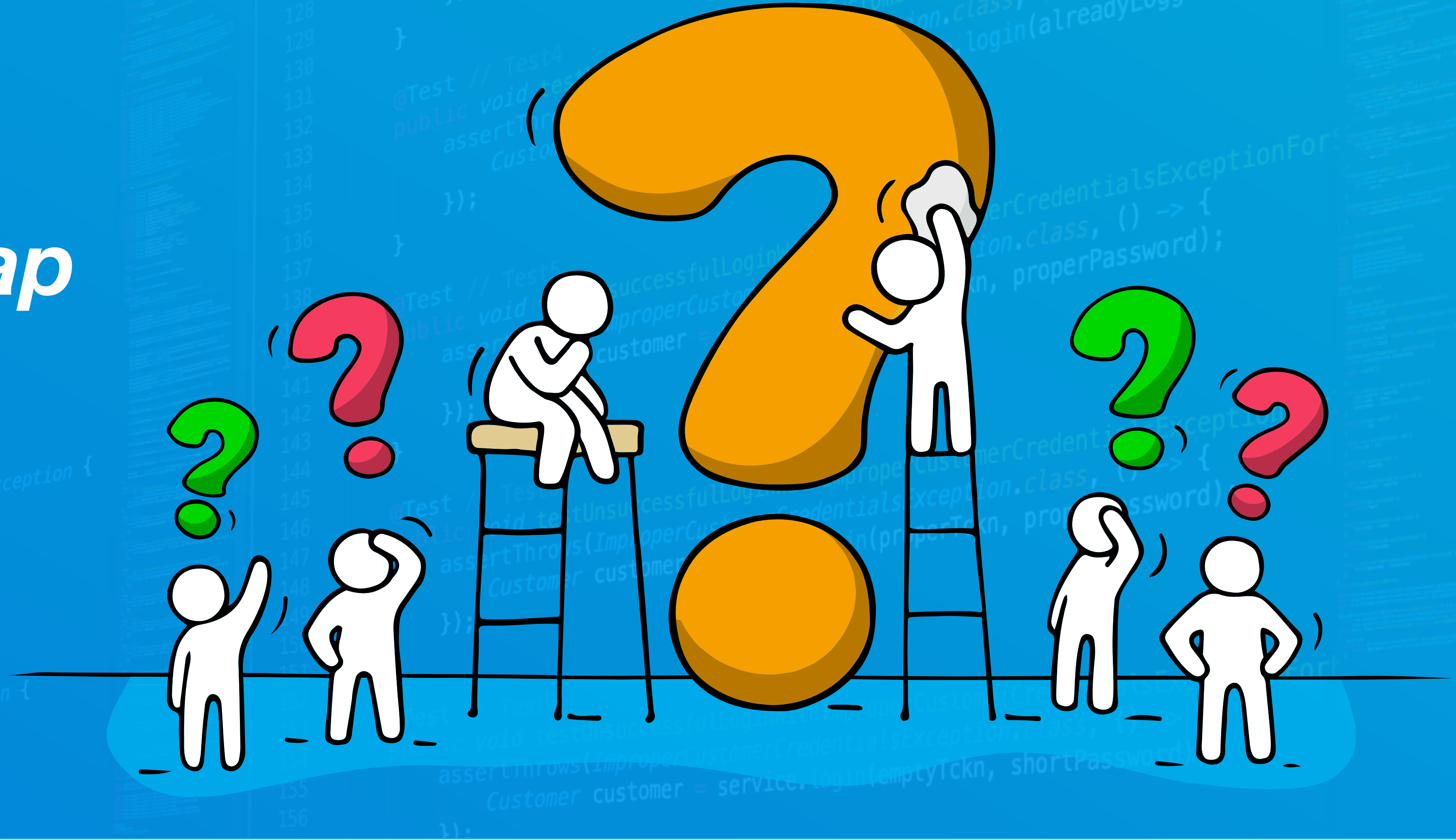
```
char a = 'a', b = 'b';  
int c = a + b;           // c ⇒ 195  
System.out.println(a + b); // 195  
System.out.println("a + b: " + 'a' + 'b'); // a + b: ab  
System.out.println('a' + 'b'); // 195
```

CharConcat



- CharConcat

Soru ve Cevap Zamanı!



Ödevler





1. **Operators** isimli bir sınıf oluşturup, içine koyacağınız **main** metoda aşağıdaki işlemcileri kullanacak şekilde, yerel değişkenlerle işlemler yapın. İşlemleri yapmak için gerekli yardımcı metotları da oluşturun.
 - a. Birleşik atamalar
 - b. Değişkenin önünde ve arkasında kullanarak arttırma ve eksiltme işlemcileri
 - c. Normal ve kısa devre halleriyle şartlı işlemciler



2. Aşağıdaki ifadelerinin kaç basacağını tahmin edin ve sonra kodu yazarak test edin

- Kodu denemek için JShell'i tercih edebilirsiniz

```
System.out.println(10 / 4);  
System.out.println(10.0 / 4);  
System.out.println((double) 10 / 4);  
System.out.println((double) (10 / 4));  
System.out.println(7 / 2);  
System.out.println(7 / 2.0);  
System.out.println(7.0 / 2);  
System.out.println((double) 7 / 2);  
System.out.println((double) (7 / 2));  
System.out.println(7 / (double) 2);
```



3. Yandaki işlemlerin sonucunu belirleyin

```
int i = 2 * 19 % 2;

int i = 5, j = 8;
int i = j++;

int i = 7;
int j = 5;
int k = i + (i > 6 ? ++j : --j);

int i = 22;
boolean b = i ≥ 12 && i < 29;

boolean b1 = false;
int y = 10;
boolean b2 = (y ≠ 5) ^ (b1 = false);
boolean b3 = (y ≠ 5) ^ (b1 = false);

int i = 5;
i += i++;
int j = i;
```



4. Yandaki kodda basılacak değerleri tahmin edin
- Sonra yazıp sonucu gözlemleyin

```
System.out.println(5 + 3 + " hello");  
System.out.println("hello " + 5 + 3);  
System.out.println(5 + " hello " + 3);  
System.out.println('A' + 'B');  
System.out.println('A' + "" + 'B');  
System.out.println("" + 'A' + 'B');  
System.out.println('A' + 1);  
System.out.println((char) ('A' + 1));  
System.out.println('Z' - 'A');  
System.out.println('a' - 'A');  
System.out.println('5' - '0');
```



5. Aşağıdaki kod parçalarında basılacak değeri tahmin edin
- Sonra yazıp sonucu gözlemleyin

```
System.out.println(10 == 10.0);  
System.out.println('2' == 50); // '2' has ASCII value 50  
System.out.println('2' == 2);  
System.out.println(0.0 == -0.0);  
System.out.println(5/2 == 2.5);  
System.out.println(5/2.0 == 2.5);
```



6. Aşağıdaki kod parçalarında basılacak değeri tahmin edin

- Sonra yazıp sonucu gözlemleyin

```
int x = 5;  
System.out.println(x++);  
System.out.println(x);  
System.out.println(++x);  
System.out.println(x);  
  
double y = 3.7;  
System.out.println((int) y++);  
System.out.println(y);
```



7. Aşağıdaki kod parçalarında basılacak değeri tahmin edin

- Sonra yazıp sonucu gözlemleyin

```
byte b = 127;  
System.out.println(b);  
System.out.println(++b);  
System.out.println(++b);  
System.out.println(b);  
  
int i = Integer.MAX_VALUE;  
System.out.println(i);  
System.out.println(i + 1);  
System.out.println(i + 2);
```

Bundan Sonra?



Bundan Sonra?



- Bundan sonraki konumuz akış kontrolüdür
- Bu bölümde tekrarlama, karar verme ve seçme yapılarını göreceğiz

Bölüm Sonu

Soru ve Cevap Zamanı!

